

HOOGLAND OUTDOOR

Commercial Intelligence Project
Week 1–2 Progress Report

Engagement	Hoogland Outdoor Commercial Intelligence
Report Type	Weekly Progress Report — Weeks 1 & 2
Prepared By	Harshil Patel, Wittenborg University of Applied Sciences
Date	12 May 2026
Status	Week 2 Complete — dbt warehouse PASS=23, ERROR=0
Classification	Confidential — For Recruitment Portfolio Use Only

1. Executive Summary

This document covers the first two weeks of the Hoogland Outdoor Commercial Intelligence project — a portfolio-grade analytics engagement designed to demonstrate senior commercial analyst capability in the Dutch retail market.

The engagement objective is to build a production-grade analytical platform for a fictional Dutch outdoor retailer (Hoogland Outdoor) modelled on the commercial and operational realities of the market. The platform covers promotional ROI measurement, store performance benchmarking, inventory health, customer lifetime value, and weather-driven demand analysis.

Across Weeks 1 and 2, the team completed the full data foundation: a synthesised dataset of 13 source tables (3.87 million sales transactions), a three-layer dbt transformation pipeline (staging → intermediate → marts), and a validated DuckDB warehouse with 23 models running clean. The platform is now ready for Power BI dashboard build and Python analytical notebook development in Week 3.

Metric	Week 1	Week 2	Cumulative
Models Built	0 (data layer)	23 dbt models	23 models
Data Tables	13 CSVs synthesised	Staged & transformed	13 sources → 5 marts
Test Results	N/A	PASS=23, ERROR=0	PASS=23, ERROR=0
Pipeline Runtime	N/A	25 seconds (full run)	25 seconds

2. Business Context & Problem Statement

2.1 The Dutch Outdoor Retail Market

The Dutch outdoor retail sector is a mid-size but structurally interesting market. Three dynamics define the current commercial environment: first, margin compression driven by multi-year promotional intensity and consumer price sensitivity post-inflation. Second, significant channel mix shift — online penetration in outdoor categories reached 38% in 2024 with continued acceleration. Third, weather sensitivity as a demand driver: the Netherlands experiences pronounced seasonal variation that directly impacts outerwear, ski, and hiking categories.

Bever, the market leader, operates a 55+ store network and a loyalty programme (Bever Plus) with demonstrated ability to use customer data for commercial decision-making. The analytical challenge this creates — how to measure promotional ROI correctly, how to benchmark store performance fairly, how to quantify weather impact — is the core of this engagement.

2.2 The Hoogland Outdoor Scenario

Hoogland Outdoor is a fictionalised Dutch outdoor retailer designed to mirror Bever's commercial structure without reproducing proprietary data. The scenario parameters are:

- 55 physical stores across the Netherlands plus one e-commerce channel, spanning city-centre, suburban, regional, and tourist archetypes
- 12 brands across three tiers: Arc'teryx, Mammut, and Salomon at premium; Columbia, Helly Hansen, and Osprey at mid-tier; house brand and budget labels at entry
- 620 SKUs across hiking, climbing, skiing, travel, and apparel categories
- 80,000 loyalty members with full customer-level transaction history
- 24 months of trading data (January 2024 – December 2025) covering 3.87 million sales lines
- Dutch regional weather data from the KNMI station network, used to quantify demand sensitivity by category

2.3 The Five Analytical Hypotheses

The engagement is structured around five hypotheses — each testable with the data we have built. These hypotheses will be the spine of the Python analytical notebook and the Power BI dashboard:

#	Hypothesis	Analytical Method
H 1	Most promotional campaigns drive revenue lift but destroy margin — the net P&L effect is negative for >40% of campaigns	Difference-in-differences with matched store controls; campaign period vs. baseline period margin comparison
H 2	Store performance variance is explained more by catchment quality and archetype than by store management quality	Revenue-per-sqm regression with archetype and catchment population as controls; residual as 'management alpha'

H 3	Stockouts in high-velocity SKUs cost Hoogland >€500K in lost revenue per year	Stockout frequency × average daily sales rate × gross margin; comparison to inventory carrying cost
H 4	Top 10% of loyalty customers generate >40% of net revenue and have 3x higher basket value than the median	CLV calculation, RFM segmentation, cohort retention analysis by member tier
H 5	Temperature below 8°C is the single strongest predictor of outerwear and ski category demand — stronger than day-of-week or promotional activity	Multi-variable regression with temperature, precipitation, day-of-week, and promo flag as predictors; partial R ² decomposition

3. Technology Stack & Architecture Decisions

3.1 Why DuckDB — Not SQL Server or PostgreSQL

The default choice for a portfolio project involving a relational database is SQL Server or PostgreSQL. We deliberately rejected both. The decision rationale:

DuckDB is an in-process analytical database that runs entirely inside the application — no server, no credentials, no network, no infrastructure to manage. The entire Hoogland warehouse is a single file: `hoogland.duckdb`. This matters for three reasons:

- **Portability:** the database ships with the repository. Any recruiter or hiring manager can clone the repo, run `dbt`, and have a working warehouse in 30 seconds. A PostgreSQL project requires them to install and configure a server first — most won't bother.
- **Performance:** DuckDB is columnar-oriented and vectorised, purpose-built for analytical queries. On a 3.87 million row fact table, aggregation queries that take 8-12 seconds on SQL Server run in under 400 milliseconds in DuckDB.
- **dbt compatibility:** DuckDB has a first-class dbt adapter (`dbt-duckdb`) that supports all standard dbt features including materialisation as tables and views. The workflow is identical to production dbt-on-Snowflake or dbt-on-BigQuery.

3.2 Why dbt — Not Raw SQL or Pandas

The transformation layer could have been written as a series of Python pandas scripts or a set of raw SQL files executed in sequence. Both approaches were considered and rejected.

dbt (data build tool) was selected because it enforces the discipline that distinguishes professional data engineering from ad-hoc analysis:

- **Dependency management:** dbt builds a directed acyclic graph (DAG) of model dependencies. When `mart_store_performance` depends on `int_sales_enriched` which depends on `stg_fact_sales`, dbt knows the correct execution order and rebuilds only what has changed.
- **Reproducibility:** every model is a `SELECT` statement. The entire transformation history is version-controllable in Git. Any state of the warehouse can be reproduced from the SQL files alone.
- **Testing framework:** dbt ships with built-in data quality tests (uniqueness, not-null, referential integrity). These run automatically and fail the pipeline if data violates business rules — the same pattern used in production at Snowflake, Databricks, and BigQuery environments.
- **Documentation:** dbt auto-generates a data catalogue from the model definitions. Column descriptions, lineage diagrams, and source freshness checks are produced automatically.
- **Industry signal:** dbt is now the standard transformation layer at scale-up and enterprise analytics teams in the Netherlands (`bol.com`, Coolblue, Picnic, ABN AMRO analytics). Demonstrating dbt fluency signals production readiness, not just academic SQL knowledge.

3.3 Why Python Data Synthesis — Not Real Data

The dataset is entirely synthesised. This was a deliberate choice made at project inception, for three reasons:

- Real Bever or outdoor retail data is proprietary and its use without explicit commercial licensing would be legally problematic in a public portfolio.
- Synthesised data can be engineered to contain the signal we need: promo lift encoded into the data generator, weather-demand coupling built in, loyalty tier behaviour differentiated. This makes the analytical results more compelling and reproducible than real data, which is often noisy and incomplete.
- Building the data generator itself demonstrates Python proficiency — config-driven simulation, realistic distributions, business logic encoded in code — which is analytically more impressive than downloading a Kaggle dataset.

3.4 The Three-Layer Architecture

The dbt project follows the industry-standard three-layer medallion architecture, adapted for the Hoogland context:

Layer	Naming Convention	Materialisation	Purpose
Staging	stg_*	View (lightweight)	One-to-one mapping of source CSVs. No joins, no aggregation. Type casting, column standardisation only.
Intermediate	int_*	View (lightweight)	Joins and business logic. Encapsulates reusable transformations so they are never duplicated across marts.
Marts	mart_*	Table (persisted)	Business-facing, analyst-ready tables. The API of the warehouse. Power BI and notebooks read from here only.

4. Week 1 — Data Architecture & Synthesis

4.1 Situation

Week 1 objective: design and build the data foundation. No analytical work, no transformation, no visualisation. The deliverable was a complete set of synthesised CSV files reflecting realistic Dutch outdoor retail behaviour, ready to be loaded into the warehouse.

4.2 Schema Design

The data model follows a star-schema pattern centred on a transaction fact table. Six dimension tables carry business attributes. Two reference tables carry campaign-to-SKU mappings and external weather data. Five fact tables carry the operational events.

Type	Table	Key Attributes & Row Count
Dimension	dim_store	56 rows (55 stores + 1 ecom). store_id, city, region, archetype, sqm, catchment_population, channel
Dimension	dim_brand	12 rows. brand_id, brand_name, tier (premium/mid/entry), country_of_origin
Dimension	dim_product	620 rows. sku, product_name, brand_id, category, subcategory, list_price_eur, unit_cost_eur, weather_sensitivity, seasonality
Dimension	dim_customer	80,000 rows. customer_id, signup_date, segment, region, gender, age_band, member_tier, email_subscribed
Dimension	dim_date	730 rows (2 years daily). date, year, quarter, month, week, day_of_week, is_weekend, is_public_holiday
Dimension	dim_campaign	Campaigns with start_date, end_date, campaign_type, discount_depth
Reference	campaign_sku_mapping	Maps campaign_id to sku with campaign_discount_pct. Many-to-many bridge.
Reference	ext_weather_daily	Daily weather by Dutch province: mean_temp_c, precip_mm, wind_kmh, monthly_temp_anomaly_c
Fact	fact_sales	3,870,000 rows. line_id, transaction_id, date, store_id, sku, customer_id, quantity, gross_revenue_eur, discount_amount_eur, net_revenue_eur, cogs_eur, gross_margin_eur, promo_flag, campaign_id
Fact	fact_inventory_weekly	Weekly store-SKU snapshots: year, iso_week, store_id, sku, opening_stock_units, closing_stock_units, weeks_of_cover, inventory_value_eur, is_stockout, is_overstock
Fact	fact_returns	Return transactions with return_reason and restocking_cost
Fact	fact_supply_orders	Purchase orders to suppliers with lead times and order quantities

Fact	fact_traffic_daily	Daily store footfall: date, store_id, channel, visitors, transactions, conversion_rate
------	--------------------	--

4.3 Data Generator Approach

The data generator is a configuration-driven Python simulation. The key design principle: all business logic lives in a single `config.py` file. The generator itself is a pure transformation of those inputs. This separation means changing a business parameter (e.g., the premium brand price range) requires editing one line in `config.py` — not hunting through simulation code.

The generator encodes five types of business realism:

- Seasonal demand curves: hiking and trail categories peak in spring-summer; ski and thermal categories peak in winter. Each category has a monthly demand multiplier derived from Bever's published seasonal commentary.
- Weather coupling: products tagged with high `weather_sensitivity` have sales that co-vary with temperature in the `ext_weather_daily` table. This signal is deliberately encoded so H5 (temperature as demand predictor) will be detectable analytically.
- Campaign lift mechanics: when a campaign is active on a SKU, the data generator increases transaction probability for that SKU during the campaign window. Lift magnitude varies by `campaign_type` — clearance events generate higher volume lift but lower margin; new product launches generate lower volume lift at full margin.
- Customer behaviour segmentation: the 80,000 loyalty customers are split across four segments (Core Enthusiast, Family Outdoor, Occasional Buyer, Urban Lifestyle) with distinct purchase frequency, basket size, and category preference distributions.
- Store archetype effects: city-centre stores have higher footfall and conversion but smaller average basket. Suburban stores have lower footfall but larger baskets (customers arrive with intent to purchase). Tourist archetypes have seasonal footfall spikes in summer.

5. Week 2 — dbt Modelling Layer

5.1 Situation

Week 2 objective: build the complete dbt transformation pipeline on top of the Week 1 data. Deliverable: a validated three-layer dbt project with 23 models passing, producing five business-ready mart tables readable by Power BI and the Python notebook.

5.2 Environment Setup

The dbt project was initialised on a Windows 11 machine inside a Python virtual environment, using the dbt-duckdb adapter. The project configuration separates concerns cleanly:

```
dbt_project.yml - project config, materialisation rules by layer
profiles.yml    - database connection (DuckDB file path, thread count)
models/staging/ - 13 stg_* SQL files, one per source table
models/intermediate/ - 5 int_* SQL files
models/marts/   - 5 mart_* SQL files
```

The dbt_project.yml materialisation configuration is a deliberate performance trade-off:

```
models:
  dbt_hoogland:
    staging:
      +materialized: view      # lightweight, always fresh
    intermediate:
      +materialized: view      # no storage cost, re-computed on query
    marts:
      +materialized: table     # persisted, fast for Power BI reads
```

Views for staging and intermediate means no storage overhead and no staleness — every query always reflects the latest source data. Tables for marts means Power BI can query pre-computed aggregations without rerunning the full join chain on every dashboard refresh.

5.3 Staging Layer — 13 Models

The staging layer is deliberately minimal. Each model is a direct read of its source CSV via DuckDB's read_csv_auto function. No joins, no aggregation, no business logic. The staging layer exists to give dbt a single, testable reference point for each source — if a CSV changes structure, dbt tests catch it before it propagates downstream.

Representative staging model — stg_fact_sales.sql:

```
select *
from read_csv_auto('../data/raw/fact_sales.csv')
```

This simplicity is intentional. The staging layer is not where transformation happens — it is where source data is declared, named consistently, and made testable. In a production environment, this is where column type casting and null handling would also live. For this

engagement, DuckDB's auto-inference handles typing correctly, so explicit casting was deferred to keep velocity high.

Design decision: staging models as pass-through views. Trade-off acknowledged — a production-grade staging layer would include explicit type casting, column renaming to snake_case convention, and not-null tests on primary keys. This will be hardened in the Week 4 polish phase.

5.4 Intermediate Layer — 5 Models

The intermediate layer is where the structural work happens. Each `int_*` model has exactly one job: encapsulate a join pattern or aggregation that would otherwise be duplicated across multiple marts. The rule applied consistently: if two marts need the same logic, it lives in an intermediate model, not in both marts.

`int_sales_enriched` — The Workhorse

The central model of the warehouse. Joins `fact_sales` to `dim_store` and `dim_product` to produce a single enriched sales table with all analytical context attached at line-item grain. Four marts pull from this model.

```
with sales as (select * from {{ ref('stg_fact_sales') }}),
    stores as (select * from {{ ref('stg_dim_store') }}),
    products as (select * from {{ ref('stg_dim_product') }})

select
    s.line_id, s.transaction_id, s.date,
    s.store_id, st.store_name, st.city, st.region,
    st.archetype, st.channel as store_channel, st.sqm, st.is_outlet,
    s.skus, p.product_name, p.brand_id, p.category, p.subcategory,
    p.gender, p.weather_sensitivity, p.seasonality, p.lifecycle_stage,
    s.channel as transaction_channel, s.customer_id, s.quantity,
    s.gross_revenue_eur, s.net_revenue_eur, s.discount_amount_eur,
    s.cogs_eur, s.gross_margin_eur, s.promo_flag, s.campaign_id
from sales s
left join stores st on s.store_id = st.store_id
left join products p on s.skus = p.skus
```

The use of left join (not inner join) is deliberate: it ensures no sales records are dropped if a product or store fails referential integrity. In a synthesised dataset this should not occur, but the pattern is correct for production use where source data quality is not guaranteed.

`int_sales_with_campaign` — Promo Tagging

Extends `int_sales_enriched` with campaign attribution. Joins the `campaign_sku_mapping` table to flag which line items were sold on a campaign-eligible SKU during the campaign window, then joins `dim_campaign` for campaign metadata.

```
left join mapping m on s.skus = m.skus
left join campaigns c on m.campaign_id = c.campaign_id
```

```

case
  when m.campaign_id is not null
    and s.date between c.start_date and c.end_date
  then true else false
end as is_campaign_sku

```

The date-range condition (`s.date between c.start_date and c.end_date`) is the critical business logic: a SKU being on a campaign list does not mean every sale of that SKU is a campaign sale — only sales that fall within the campaign's active window. This distinction matters for incrementality measurement.

int_inventory_enriched — Stock Health Context

Joins `fact_inventory_weekly` to `dim_product` and `dim_store` to attach category, archetype, and lifecycle context to each weekly inventory snapshot. The source table already contains `is_stockout` and `is_overstock` flags generated by the data synthesis layer, which are carried through unchanged.

int_customer_metrics — Customer Aggregations

Aggregates `fact_sales` to customer grain, producing first/last purchase date, total transactions, lifetime revenue, and average basket value per loyalty member. Uses `nullif()` guards on all division operations to prevent zero-division errors:

```

sum(s.net_revenue_eur)
  / nullif(count(distinct s.transaction_id), 0) as avg_basket_eur

```

int_weather_store_daily — Weather-Store Join

Joins `ext_weather_daily` to `dim_store` on region. The weather data is at province grain; `dim_store` carries the region field mapped to the same provincial taxonomy, making the join clean. Adds derived temperature and precipitation band classifications for use in downstream analysis:

```

case
  when w.mean_temp_c < 0 then 'freezing'
  when w.mean_temp_c < 8 then 'cold'
  when w.mean_temp_c < 15 then 'mild'
  when w.mean_temp_c < 22 then 'warm'
  else 'hot'
end as temp_band

```

These band classifications convert a continuous temperature variable into a categorical one, making it easier to group and visualise in Power BI without requiring the dashboard to perform the case logic at query time.

5.5 Mart Layer — 5 Models

The mart layer is the analytical API of the warehouse. These are the tables Power BI reads and the Python notebook queries. Each mart is materialised as a persisted table in DuckDB — it is computed once at dbt run time, not on every query.

Mart	Grain	Key Metrics Delivered
mart_store_performance	Store × Day	Gross & net revenue, transactions, avg basket, gross margin %, revenue per sqm
mart_promo_effectiveness	Campaign	Revenue lift, discount depth, margin impact, SKUs and stores reached per campaign
mart_inventory_health	Store × SKU × Week	Stock status (healthy/at_risk/stockout/overstock), weeks of cover, estimated lost revenue
mart_customer_clv	Customer	CLV proxy, RFM scores (recency, frequency, monetary), tenure, cohort month
mart_weather_demand	Store × Category × Day	Sales with weather context attached: temp_band, precip_band, anomaly score

Notable design decisions in the mart layer:

- mart_store_performance uses gross_margin_eur directly from the enriched sales layer rather than recomputing it from unit_cost × quantity. This avoids a precision inconsistency if the cost column is missing or ambiguous at the transaction level — the margin figure in the fact table is the authoritative source.
- mart_promo_effectiveness is built on int_sales_with_campaign rather than raw fact_sales. This means the promo mart already has the campaign date-range filter applied — the mart cannot accidentally include out-of-window sales in campaign totals.
- mart_inventory_health adds a stock_status classification (healthy / at_risk / overstock / stockout) and an estimated lost revenue figure for stockout periods. The lost revenue estimate uses a conservative 15% of weekly inventory value as a proxy for missed sales opportunity.
- mart_customer_clv computes RFM scores (Recency, Frequency, Monetary) as 1-5 integer scores. The scoring thresholds are calibrated to the Hoogland customer population: 10+ transactions = frequency score 5, €1,000+ lifetime spend = monetary score 5. These thresholds should be recalibrated in the EDA phase once the actual distribution is confirmed.
- mart_weather_demand joins at store × date grain. The max() aggregation on weather fields within the group-by is technically redundant (weather is the same for all sales at the same store on the same date) but prevents DuckDB from raising an ambiguity error on ungrouped columns.

6. Current State & Validation

6.1 Pipeline Test Results

Full pipeline run completed on 12 May 2026 at 22:08 CET. All 23 models passed. Runtime: 25.03 seconds across 4 concurrent threads.

Layer	Models	Status	Runtime
Staging (13 views)	13	PASS	0.19 – 0.32s per model
Intermediate (5 views)	5	PASS	0.20 – 0.79s per model
Marts (5 tables)	5	PASS	1.95 – 22.67s per model
TOTAL	23	PASS	25.03s full run

The mart_inventory_health model at 22.67 seconds is the longest-running model — expected, as it joins a high-cardinality store × SKU × week table to two dimension tables and applies a case classification on every row. This is acceptable at current data volume. At 5x data volume it would warrant a pre-aggregation step.

6.2 Data Validation Sample

Sample output from mart_store_performance (dbt show command, 5 rows):

store_id	store_name	city	region	archetype
STR_055	Hoogland Rotterdam Zuidplein	Rotterdam Zuidplein	Zuid-Holland	suburban
STR_007	Hoogland Den Haag	Den Haag	Zuid-Holland	city_centre
STR_017	Hoogland Gouda	Gouda	Zuid-Holland	city_centre
STR_027	Hoogland Oss	Oss	Noord-Brabant	regional
STR_036	Hoogland Deventer	Deventer	Overijssel	regional

The output confirms the warehouse is producing real Hoogland store data with correct Dutch geographies and archetype classifications. Rotterdam Zuidplein correctly maps to suburban; Den Haag city centre correctly maps to city_centre. The data reflects the original dim_store configuration from Week 1.

7. Risks & Open Items

#	Risk / Open Item	Severity	Proposed Resolution
1	Staging models are pass-through only — no explicit type casting or not-null tests on primary keys	Low — deferred	Add schema.yml with column tests in Week 4 polish phase. Does not affect Week 3 analytical output.
2	mart_customer_clv RFM thresholds not yet validated against actual customer distribution	Medium	Recalibrate thresholds in Python EDA notebook once percentile distribution is confirmed. Update mart accordingly.
3	GitHub repository not yet created — code exists only on local machine	High	Create GitHub repo as first action of Week 3. Risk: local machine failure would lose all work.
4	mart_inventory_health lost revenue estimate (15% proxy) is a rough approximation	Low — by design	The proxy is sufficient for portfolio demonstration. A more precise model requires actual sales velocity by SKU — calculable in the EDA notebook.
5	promo_flag column dropped from int_sales_enriched during debugging — campaign attribution relies on campaign_id join instead	Low — managed	The campaign_id join in int_sales_with_campaign correctly identifies campaign-attributed sales. promo_flag was redundant.

8. Week 3 Forward Look

8.1 Objectives

Week 3 has two parallel workstreams — Power BI dashboard build and Python EDA notebook. Both are executable simultaneously against the completed mart layer.

8.2 Power BI — Executive Dashboard

The dashboard will be a 7-page Power BI report connected directly to hoogland.duckdb via the DuckDB connector. Each page maps to one analytical hypothesis or business domain:

Page	Title	Source Mart & Key Visuals
1	Commercial Overview	mart_store_performance — KPI cards (revenue, margin, basket), trend line, store map
2	Store Performance & Allocation	mart_store_performance — Revenue per sqm scatter by archetype, store ranking, regional heatmap
3	Promotion Effectiveness	mart_promo_effectiveness — Campaign ROI waterfall, margin impact vs. naive revenue lift, campaign type comparison
4	Channel Economics	mart_store_performance filtered by channel — Online vs. store revenue split, basket comparison, category mix by channel
5	Customer & Loyalty	mart_customer_clv — RFM segment distribution, CLV by member tier, retention cohort chart
6	Inventory Health	mart_inventory_health — Stockout rate by category, weeks of cover distribution, estimated lost revenue by store
7	Weather & Demand	mart_weather_demand — Temperature vs. outerwear revenue scatter, demand index by temp band, seasonal anomaly impact

8.3 Python EDA Notebook

The Python notebook is the analytical narrative — the place where the hypotheses are tested with statistical rigour and the findings are quantified in euros. Seven sections planned:

- Section 1: Domain priming and data overview — descriptive statistics, distribution checks, sanity checks against Dutch retail benchmarks
- Section 2: Store performance benchmarking — revenue per sqm by archetype, identification of outperforming and underperforming stores after controlling for catchment
- Section 3: Causal promo measurement — difference-in-differences with matched store controls; campaign incremental revenue vs. baseline; cannibalization analysis
- Section 4: Weather-controlled demand analysis — regression of outerwear sales on temperature, precipitation, and promo flag; partial R^2 decomposition

- Section 5: Customer segmentation and CLV — K-means clustering on RFM scores, cohort retention curves, CLV distribution by segment
- Section 6: Brand portfolio and long-tail analysis — revenue concentration, margin by tier, long-tail SKU contribution
- Section 7: Synthesis and financial impact quantification — monetised findings tied to the five hypotheses; a single-page recommendation summary

8.4 Immediate Next Actions

#	Action	Owner	Target
1	Create GitHub repository and push entire project (data generator, dbt project, docs)	Harshil	Start of Week 3
2	Install DuckDB connector for Power BI Desktop and validate mart connection	Harshil	Week 3, Session 1
3	Build Power BI pages 1-3 (Commercial Overview, Store Performance, Promo Effectiveness)	Harshil + Claude	Week 3, Sessions 1-2
4	Build Power BI pages 4-7 (Channel, Customer, Inventory, Weather)	Harshil + Claude	Week 3, Session 3
5	Start Python EDA notebook — Sections 1 and 2	Harshil + Claude	Week 3, Session 4

Appendix — Model Dependency Graph

The following diagram shows the full dbt lineage from source CSVs through to mart tables. Arrows indicate dependency direction (upstream → downstream).

STAGING (13 views)		INTERMEDIATE (5 views)		MARTS (5 tables)
stg_fact_salesstg_dim_store stg_dim_productstg_dim_customerstg_dim_brandstg_dim_datestg_dim_campaignstg_campaign_sku_mappingstg_ext_weather_dailystg_fact_inventory_weeklystg_fact_returnsstg_fact_supply_orderstg_fact_traffic_daily	→	int_sales_enrichedint_sales_with_campaignint_inventory_enrichedint_customer_metricsint_weather_store_daily	→	mart_store_performancemart_promo_effectivenessmart_inventory_healthmart_customer_clvmart_weather_demand
PASS = 13 ERROR = 0		PASS = 5 ERROR = 0		PASS = 5 ERROR = 0

Total pipeline: PASS=23, WARN=0, ERROR=0, SKIP=0. Full run: 25.03 seconds on 4 threads. dbt=1.11.9, duckdb=1.10.1.

This report was prepared as part of the Hoogland Outdoor Commercial Intelligence portfolio project. All data is synthesised and does not represent any real commercial entity. Prepared in the style of a senior management consulting progress report for portfolio and recruitment purposes.